

Aim: Study of OWASP to top 10 security risk or vulnerabilities by OWASP

Introduction

The Open Worldwide Application Security Project (OWASP) is a non-profit organization that works to improve software and web application security. The OWASP Top 10 is a widely recognized awareness document that lists the most critical security risks faced by web applications.

It helps developers, testers, and security professionals identify, understand, and reduce common vulnerabilities in web applications.



OWASP Top 10 Security Risks

No.	Vulnerability	Description
1	Broken Access Control	Users can access resources or perform actions they should not be authorised to access.
2	Cryptographic Failures	Sensitive data is exposed due to weak or missing encryption.
3	Injection	Untrusted data is sent to an interpreter, causing unintended commands or queries to execute.
4	Insecure Design	Security weaknesses exist because security was not properly considered during the design phase.
5	Security Misconfiguration	Incorrect security settings, default passwords, unnecessary services, or improperly configured systems create vulnerabilities.

6	Vulnerable and Outdated Components	Using software libraries or components with known security vulnerabilities.
7	Identification and Authentication Failures	Weak login, authentication, password, or session management allows attackers to compromise accounts.
8	Software and Data Integrity Failures	Software or data is trusted without properly verifying its integrity or source.
9	Security Logging and Monitoring Failures	Insufficient logging and monitoring prevent the detection and investigation of attacks.
10	Server-Side Request Forgery (SSRF)	An attacker tricks a server into making requests to unintended internal or external resources.

1. Broken Access Control

This occurs when an application does not properly enforce permissions. For example, a normal user may be able to access an administrator page simply by changing a URL.

Prevention:

- Enforce authorisation on the server side.
- Deny access by default.
- Check permissions for every sensitive request.

Scenario:

A website has a user profile page: `example.com/profile?id=101`

A user changes the ID to 102 and can view another user's private profile information without permission.

Problem: The application does not properly verify whether the logged-in user is authorized to access the requested data.

Risk: Unauthorized access to personal data, admin functions, or confidential files.



2. Cryptographic Failures

Sensitive information such as passwords, credit card details, or personal data may be exposed when encryption is weak or missing.

Prevention:

- Use strong modern encryption.
- Protect sensitive data during storage and transmission.
- Never store passwords in plain text.

Scenario:

An online shopping website stores user passwords in plain text. If the database is stolen, attackers can immediately view all user passwords.

Problem: Sensitive information is not properly encrypted or securely protected.

Risk: Exposure of passwords, financial information, personal data, or confidential communications.



3. Injection

Injection attacks occur when untrusted input is interpreted as commands or code. Examples include SQL Injection and command injection.

Prevention:

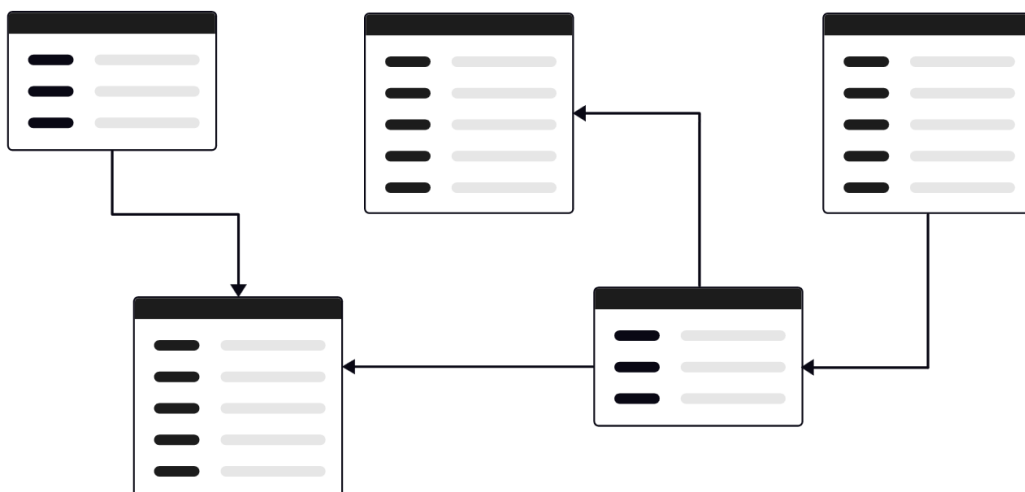
- Use parameterised queries.
- Validate and sanitise input.
- Avoid dynamically constructing commands from user input.

Scenario:

A login form accepts user input and directly uses it in a database query. An attacker enters specially crafted input that changes the intended query and accesses unauthorized database information.

Problem: The application does not properly validate user input before processing it as part of a command or query.

Risk: Unauthorized database access, data theft, data modification, or deletion.



4. Insecure Design

A system may have security problems because security controls were not considered during its design.

Prevention:

- Perform threat modelling.
- Define security requirements before development.
- Design secure workflows and access controls.

Scenario:

A banking application allows users to reset their password using only their username and date of birth. An attacker who knows this information can reset the user's password.

Problem: The application has a security weakness built into its fundamental design.

Risk: Account takeover, identity theft, and unauthorized access to sensitive information.



5. Security Misconfiguration

Examples include default passwords, unnecessary services, verbose error messages, and incorrectly configured cloud or server settings.

Prevention:

- Use secure default configurations.
- Remove unnecessary features and services.
- Regularly review security settings.

Scenario:

A web server is deployed with a default administrator username and password. It also displays detailed error messages containing internal system information.

Problem: The application or server is configured with insecure settings.

Risk: Attackers may gain unauthorized access or obtain information useful for further attacks.

6. Vulnerable and Outdated Components

Applications often depend on third-party libraries. If these components contain known vulnerabilities, the application may also become vulnerable.

Prevention:

- Keep dependencies updated.
- Track third-party components.
- Remove unused libraries.

Scenario:

A website uses an old third-party software library that contains a publicly known security vulnerability. The website owner does not update the library.

Problem: The application depends on software components with known security weaknesses.

Risk: Attackers may exploit the vulnerable component to compromise the application or server.



7. Identification and Authentication Failures

Weak passwords, poor session management, and ineffective authentication can allow attackers to take over accounts.

Prevention:

- Use strong password policies.
- Implement multi-factor authentication.
- Properly manage sessions and login attempts.

Scenario:

A website allows users to set weak passwords and does not limit repeated login attempts. An attacker repeatedly guesses passwords until successfully logging into an account.

Problem: The application has weak authentication and account protection mechanisms.

Risk: Account takeover, identity theft, and unauthorized access.



8. Software and Data Integrity Failures

This risk occurs when applications trust software updates, plugins, or data without verifying their authenticity and integrity.

Prevention:

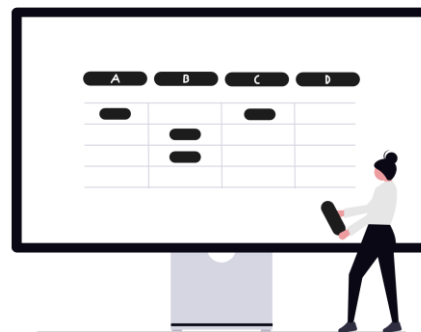
- Verify software sources.
- Use secure update mechanisms.
- Check the integrity of important data.

Scenario:

An application automatically downloads software updates without verifying their source or integrity. An attacker compromises the update source and distributes a modified update.

Problem: The application trusts software or data without properly verifying its authenticity and integrity.

Risk: Malicious code may be installed, leading to data theft or system compromise.



9. Security Logging and Monitoring Failures

Without proper logging, attacks may remain unnoticed for a long time.

Prevention:

- Log important security events.
- Monitor suspicious activity.
- Protect logs from unauthorized modification.

Scenario:

An attacker repeatedly attempts to log into an administrator account. The application does not record the failed login attempts or alert the administrator.

Problem: Important security events are not properly logged, monitored, or reported.

Risk: Attacks may continue for a long time without being detected or investigated.



10. Server-Side Request Forgery (SSRF)

SSRF occurs when an attacker manipulates a server into sending requests to resources that the attacker normally cannot access.

Prevention:

- Validate and restrict URLs.
- Use allows lists for trusted destinations.
- Restrict server network access.

Scenario:

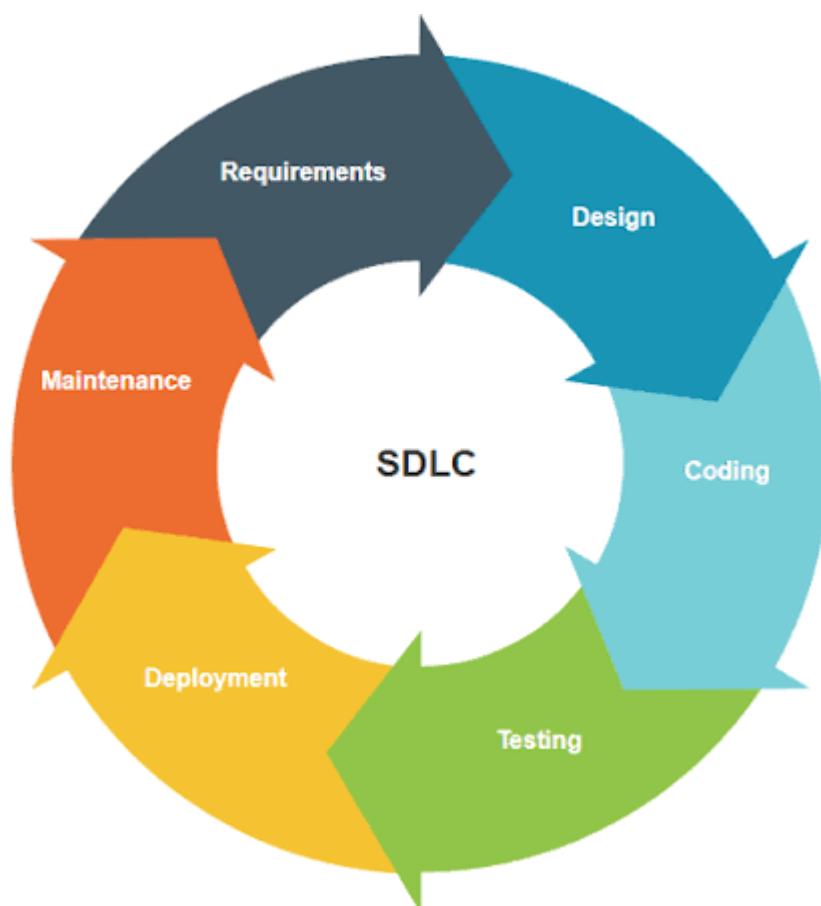
A web application allows users to enter a URL to import an image. The server fetches any URL provided by the user without checking whether it is safe.

Problem: The server can be tricked into making requests to unintended internal or external resources.

Risk: Unauthorized access to internal systems, sensitive services, or confidential information.

Threat modelling is most effective when integrated into the Secure Development Life Cycle (SDLC). The flowchart illustrates how each SDLC phase aligns with OWASP Top 10 risks:

- **Requirements Gathering:** Identify critical assets (credentials, payment data). Map potential OWASP risks like Broken Access Control and Authentication Failures.
- **Design & Architecture:** Apply secure design principles. Consider Insecure Design and Cryptographic Failures.
- **Development:** Implement secure coding practices. Prevent Injection and Security Misconfiguration.
- **Testing:** Conduct penetration testing and vulnerability scanning. Detect Logging Failures and Mishandling of Exceptions.
- **Deployment:** Harden configurations, patch components, and verify supply chain integrity.
- **Monitoring & Review:** Continuously log, monitor, and remediate risks. Ensure resilience against evolving threats.



Conclusion

OWASP Top 10 is an important resource for learning about web application security. Vulnerabilities such as broken access control, injection, cryptographic failures, authentication failures, and security misconfiguration can seriously affect the confidentiality, integrity, and availability of applications. Following secure development practices and regularly testing applications can significantly reduce these risks.